

CSS Grid - Master Revision Notes

Selva's hard copy - Module 10 (Display Grid - Sizing - Placement - Grid Garden 28/28 - Mondrian Project) - full notes in [.github/my_tasks/](https://github.com/selvasek/my_tasks/)

The 8 lines that answer 90% of Grid questions

1	<code>display: grid</code> goes on the container ; direct children become grid items.
2	Flexbox = one dimension . Grid = rows and columns together . Real sites nest both.
3	<code>grid-template-columns</code> / <code>-rows</code> draw the map; item placement gives boxes addresses.
4	<code>fr</code> is a ratio of available space , not a pixel size. Fixed tracks and gaps are removed first.
5	A grid with n tracks has n + 1 lines . Count lines, not boxes.
6	Line numbers answer where ; <code>span</code> answers how many tracks .
7	<code>grid-area</code> = row-start / column-start / row-end / column-end.
8	Debug size first: Grid is greedy sideways, shy downwards . <code>1fr</code> of no height is no height.

A The property map - container or item?

CONTAINER (parent) - creates the map

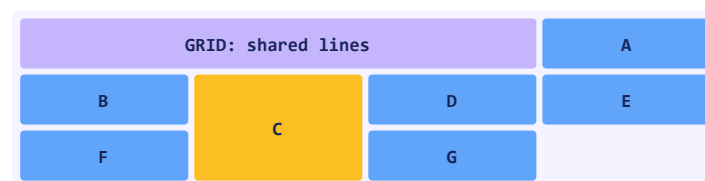
Property	Job
<code>display: grid</code>	switch Grid on
<code>grid-template-columns</code>	column count and widths
<code>grid-template-rows</code>	row count and heights
<code>grid-template</code>	shorthand: rows / columns
<code>grid-auto-rows</code>	size implicit rows
<code>grid-auto-columns</code>	size implicit columns
<code>gap</code>	gutter between tracks

ITEM (child) - chooses an address

Property	Job
<code>grid-column-start/end</code>	vertical start/end lines
<code>grid-row-start/end</code>	horizontal start/end lines
<code>grid-column</code>	column start / end shorthand
<code>grid-row</code>	row start / end shorthand
<code>grid-area</code>	four-line rectangle
<code>order</code>	visual auto-placement sequence
<code>z-index</code>	layer order when areas overlap

B Grid vs Flexbox - choose the layer, not a winner

	Flexbox	Grid
Dimensions	1D: row <i>or</i> column	2D: rows <i>and</i> columns
Instinct	squish and adapt	keep shared lines aligned
Use	nav, card row, centering	page regions, gallery, dashboard
Together	Grid builds a dashboard cell; Flexbox aligns content inside it.	



Say it your way

Flexbox = one line. Grid = a table. If the design has an X and a Y, start with Grid. Then use Flexbox inside a cell when the smaller problem is one-dimensional.

C Turning Grid on - tracks, fr, gap

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 2fr;  
  grid-template-rows: 1fr 1fr;  
  gap: 10px;  
}
```

- One space-separated value = one track.
- `1fr 2fr` makes two columns in a 1:2 ratio.
- `gap` spaces rows and columns; it is the same property from Flexbox.
- Grid items stretch to fill their cell on both axes by default.

1fr

2fr

fr is ratio thinking

In a 600px container with no gap, `1fr 2fr` becomes 200px + 400px. Same mental model as `flex:1` and `flex:2`.

Grid infers rows

Declare 8 columns and add 64 items: Grid fills items 1-8, then creates an implicit row for 9-16, and continues.

D The width/height asymmetry - the blank-page trap

Axis	Grid container default	Result
Width	block-level, fills parent	<code>fr</code> columns have space
Height	only tall enough for content	empty content can mean 0px

Selva's actual EX 10.0 bug

`grid-template-rows: repeat(8,1fr)` plus no container height and empty divs produced 0px rows - a completely blank page.

```
/* Fix A: size the board */  
.board { width:800px; height:800px; }
```

```
/* Fix B: size the squares */  
.white, .black {  
  width:100px; height:100px;  
}
```

Memory line

fr of nothing is nothing. Check the container's size on the affected axis before blaming Grid.

E EX 10.0 Chessboard - two correct solutions

Official: size children	Selva: size container
<pre>.container{width:800px} .cell{width:100px;height:100px}</pre>	<pre>.container{width:800px; height:800px; grid-template-columns: repeat(8,1fr); grid-template-rows: repeat(8,1fr); }</pre>
Rows grow from 100px child content.	Rows divide the known 800px
Valid	More idiomatic Grid: size stretch.

Why container width matters

Without `800px`, the cells' tracks can be wider than the fixed 100px squares. The whitespace belongs to oversized cells, not to `gap`.



```
.container {  
  width: 800px;  
  height: 800px;  
  display: grid;  
  grid-template:  
    repeat(8,1fr) /  
    repeat(8,1fr);  
}
```

EX 10.0 - 10/10

GRID SIZING - choose each track's behaviour

Fixed - auto - fr - minmax() - repeat() - then test the layout by resizing and changing content.

F The sizing toolbox

Tool	Meaning	Responsive to window?	Use
px	fixed track	No	hard measurement
rem	root-font-relative fixed track	No - reacts to zoom/root font, not width	scalable fixed size
auto	automatic size	axis-dependent	leftover width or content height
fr	share of available space	Yes	ratios
minmax(a,b)	floor + ceiling	Yes, clamped	flexible but protected track
repeat(n,x)	repeat x, n times	same as x	remove repetition

G auto - same word, different axis

```
grid-template-columns: 200px auto;
/* auto column eats leftover WIDTH */

grid-template-rows: 100px auto;
/* auto row fits its CONTENT height */
```

200px

auto - leftover

The lesson's main trap

Auto column stretches; auto row just fits. This is the same width-greedy / height-shy asymmetry as the blank chessboard, now behind an explicit keyword.

An auto row grows from one text line to a full paragraph, but it does not stretch merely because the viewport is tall.

H fr, minmax(), and repeat()

fr is a live ratio

1fr 2fr protects 1:2. Add lots of content to row 2 and row 1 may also grow, even when row 1's content never changed.

minmax(min,max)

Grid's one-function version of Flexbox's min-width + max-width: grow to the ceiling, shrink to the floor, never cross either.

repeat(count,value)

Works for any value:
repeat(8,1fr),
repeat(2,200px), or repeated minmax() tracks.

Clamp examples

Rule	Unclamped wish	Final
minmax(400px,800px)	300px	400px - floor
	550px	550px
	900px	800px - ceiling

grid-template shorthand

```
grid-template: 100px 200px / 400px 800px;
/* rows first / columns second */
```

Learning rule

Recognise it, but prefer explicit -rows and -columns until debugging the two axes feels automatic.

I Explicit vs implicit tracks - when item counts do not match

Too few items

Template	Items	Outcome
2 rows x 3 cols = 6 cells	4	items fill row 1, then row 2 cell 1; final 2 cells stay empty

1	2	3
4		

Too many items

Axis	Implicit-row default
Width	follows existing column tracks
Height	fits the new item's content

```
.container {  
  grid-template: repeat(2,200px) /  
  repeat(2,200px);  
  grid-auto-rows: 150px; /* take control */  
}
```

J EX 10.1 Grid Sizing - the exact specification

Track	Requirement
Rows 1-3	1fr 1fr 2fr
Implicit row 4	grid-auto-rows: 50px
Column 1	auto - fill leftover width
Column 2	400px fixed
Column 3	minmax(200px,500px)

Hands-on still to attempt

Write this yourself in the local Test page before using the worked solution.

```
.grid-container {  
  display: grid;  
  grid-template-rows: 1fr 1fr 2fr;  
  grid-template-columns:  
    auto 400px minmax(200px,500px);  
  grid-auto-rows: 50px;  
}
```

Silent typo trap

The real properties end in `s`: `grid-template-rows` and `grid-template-columns`. Unknown singular spellings are ignored with no visible error message.

K DevTools - see the grid the browser computed

1. Inspect the grid container.
2. Click the small **grid** badge in Elements.
3. Open the Layout pane.
4. Enable line numbers, extend grid lines, and **Show track sizes**.
5. Read exact computed pixel sizes instead of guessing.

Debug in this order

1. Is Grid on? 2. Does this axis have size? 3. Are property names valid? 4. Which tracks are explicit vs implicit? 5. What does the overlay report?

Memory line

Too few = empty cells. Too many = implicit tracks.
Use `grid-auto-*` when dynamic content must be predictable.

GRID PLACEMENT - give every item an address

The template draws the map. Lines, spans, shorthands, and areas choose the occupied rectangle.

L Tracks vs lines - count the borders



Three columns create four vertical lines. An item covering columns 1 and 2 uses `grid-column: 1 / 3`.

Syntax	Read it as
2 / 5	start at line 2, stop at line 5
5 / 2	same area, reverse numeric direction
-3 / -1	final two tracks from the far edge
2 / span 3	start line 2, occupy 3 tracks

Where vs how many

Line numbers answer where. span answers how many. span is a positive track count; negative numbers are for line addresses from the far edge.

M Longhand and the two-axis shorthands

```
.hero {
  grid-column-start: 2;
  grid-column-end: 5;
  grid-row-start: 1;
  grid-row-end: 3;
}
```

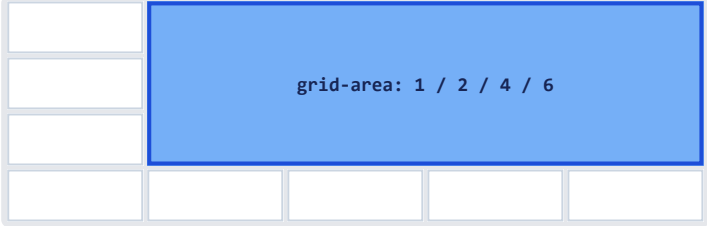
```
.hero {
  grid-column: 2 / 5;
  grid-row: 1 / 3;
}

.book { grid-row: span 2; }
```

N grid-area - the four-number rectangle

```
grid-area:
  row-start / column-start /
  row-end   / column-end;

.water {
  grid-area: 1 / 2 / 4 / 6;
}
```



Mnemonic

Row, column, row, column - walk around the rectangle. Say the labels before typing values.

Value	Address
1	row start
2	column start
4	row end
6	column end

O Placement decision table

I know...	Use	Example
start and destination line	<code>grid-column / grid-row</code>	2 / 5
start and number of tracks	line + span	2 / span 3
only the size; auto-placement can choose start	span n	<code>grid-row: span 2</code>
all four rectangle edges	<code>grid-area</code>	2 / 1 / 3 / 3
the far edge but not track count	negative lines	-3 / -1

P Auto-placement, order, and overlap

order changes the visual queue

```
.water { order: 0; } /* default */
.poison { order: 1; } /* after */
.poison { order: -1; } /* before */
```

Accessibility warning

Visual order does not change DOM reading order or keyboard focus order. Keep meaningful content order correct in the HTML.

Explicit items may overlap

```
.a { grid-area: 1 / 1 / 3 / 3; }
.b {
  grid-area: 2 / 2 / 4 / 4;
  z-index: 2;
}
```

Grid allows both claims. Source/paint order and z-index decide the visible layer.

Q Grid outside, Flexbox inside

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1.5fr;
  grid-template-rows: 1fr 1fr;
}
.item {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Different layers, different jobs

Grid decides the emoji boxes' rows, columns, and spans. Flexbox centers one emoji inside each box. Neither tool replaces the other.

Question	Answer
Where is the box?	Grid item placement
How wide/tall is it?	Grid tracks + spans
Where is content inside it?	Flexbox alignment

R EX 10.2 source audit - preserve honest status

File	Goal	Status	Reason / next action
exercise1.html	center every emoji	Correct	Flexbox centering is present.
exercise2.html	astronaut fills row 2 cols 1-2	One line missing	order:1 changes sequence; add grid-column:span 2 for width.
exercise3.html	book spans both rows on right	Correct	grid-row:span 2 is valid.

The debugging lesson from exercise 2

Position, size, and internal alignment are separate questions. order cannot change a span; span cannot center content; Flexbox centering cannot choose a grid line.

S Three placement recipes

Header across all

```
.header { grid-column:1 / -1; }
```

Right sidebar

```
.side { grid-column:-3 / -1; }
```

Known rectangle

```
.hero { grid-area:2 / 2 / 5 / 6; }
```

GRID GARDEN - verified solutions 1-14

Completed in the App Brewery course version. Every accepted rule advanced to the next numbered level.

T Lines, negative lines, span, rows, and shorthands

#	What the level drills	Verified CSS	Memory check
1	start at vertical line 3	grid-column-start: 3;	Start/end values name grid lines .
2	start at vertical line 5	grid-column-start: 5;	
3	end at vertical line 4	grid-column-end: 4;	
4	end before the start number	grid-column-end: 2;	End need not be numerically larger.
5	end from the right edge	grid-column-end: -2;	-1 is the final line.
6	start from the right edge	grid-column-start: -3;	
7	span 2 toward the end	grid-column-end: span 2;	span is a positive track count.
8	span 5 toward the end	grid-column-end: span 5;	
9	span 3 backward from the end	grid-column-start: span 3;	
10	column shorthand	grid-column: 4 / 6;	start / end
11	column shorthand + span	grid-column: 2 / span 3;	where / how many
12	row start line	grid-row-start: 3;	same idea on vertical axis
13	row shorthand	grid-row: 3 / 6;	row start / row end
14	place on both axes	grid-column: 2; grid-row: 5;	one value = one-track default span

Levels 1-6: addresses

Positive lines count from left/top. Negative lines count from right/bottom. Either numeric direction is valid.

Levels 7-14: size + two axes

span measures tracks. Column and row shorthands transfer the same start/end idea across both axes.

T+ Fast recall drill

Prompt	Answer
last line	-1
start at 2, cover 3	2 / span 3
columns between 2 and 5	2 / 5
row starts at 3	grid-row-start:3

Say it in six seconds

Where? Lines. How many? Span. Other wall? Negative. Two axes? Column + row.

GRID GARDEN - verified solutions 15-28

Areas - order - percentages - mixed units - fr arithmetic - grid-template shorthand - final win.

U Areas, ordering, and track design

#	What the level drills	Verified CSS	Memory check
15	large 2D area, two shorthands	<code>grid-column: 2 / 6;</code> <code>grid-row: 1 / 6;</code>	column + row
16	four-value area	<code>grid-area: 1 / 2 / 4 / 6;</code>	row / col / row / col
17	second overlapping area	<code>grid-area: 2 / 3 / 5 / 6;</code>	overlap is allowed
18	poison after normal items	<code>order: 1;</code>	visual queue only
19	poison before normal items	<code>order: -1;</code>	
20	first column = half	<code>grid-template-columns: 50%;</code>	one explicit column
21	8 equal percentage columns	<code>repeat(8, 12.5%)</code>	8 x 12.5 = 100
22	mixed units	<code>100px 3em 40%</code>	mixing is valid
23	whole width split 1:5	<code>1fr 5fr</code>	6 total shares
24	fixed edges, flexible middle	<code>50px 1fr 1fr 1fr 50px</code>	subtract 100px first
25	75px fixed, remainder 3:2	<code>75px 3fr 2fr</code>	5 flexible shares
26	5 rows; only row 5 fills remainder	<code>grid-template-rows: 50px 0 0 0 1fr;</code>	zero tracks intentional
27	rows / columns shorthand	<code>grid-template: 60% 40% / 200px;</code>	rows before slash
28	final garden: 2 rows x 2 cols	<code>grid-template: 1fr 50px / 20% 1fr;</code>	WIN verified

V Subtract first, share second

```
/* 1075px container, no gap */
grid-template-columns: 75px 3fr 2fr;
```

1075 - 75 = 1000px remaining
3fr + 2fr = 5 shares
1000 / 5 = 200px per share
final = 75px | 600px | 400px

75

3fr

2fr

Gap-aware version

Subtract every fixed track **and every gap** before dividing remaining space by total `fr` shares.

Completion evidence

All 28 submitted rules were accepted; each successful rule advanced to the next level; level 28 produced the final **You win!** screen and carrot-cake illustration on 2026-08-29.

MONDRIAN PROJECT - design to Grid

Capstone: fixed track geometry - black gaps - spans/areas - colour ownership - Flexbox centering.

W Frame geometry and the black-line trick

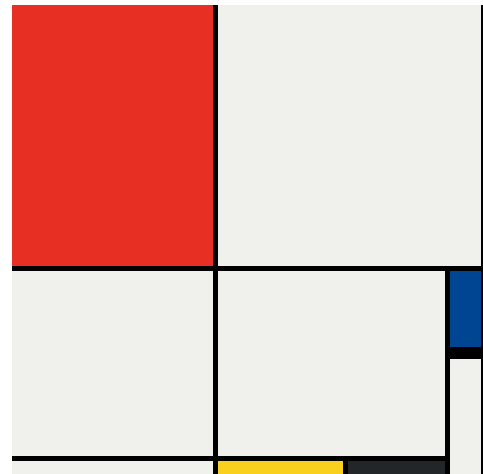
```
.container {  
  width: 748px; height: 748px;  
  display: grid;  
  background: #000;  
  grid-template-columns:  
    320px 198px 153px 50px;  
  grid-template-rows:  
    414px 130px 155px 22px;  
  gap: 9px;  
}
```

Width: $320 + 198 + 153 + 50 + (3 \times 9) = 748$.

Height: $414 + 130 + 155 + 22 + (3 \times 9) = 748$.

Why gaps become black lines

Grid lines are coordinates, not paintable objects. `gap` is transparent, so the black container background shows between the coloured items.



X Merging the nine visual regions

```
.item { background: #F0F1EC; }  
.white1 { grid-column: span 3; }  
.white2 { grid-row: span 2; }  
.white3 { grid-area: 2 / 2 / 4 / 4; }  
.white4 { grid-row: span 2; }
```

Colour	Hex
off-white	#F0F1EC
red	#E72F24
blue	#004592
yellow	#F9D01E
dark block	#232629

Normal seams belong to the container gap. The special line below blue belongs to `.blue { border-bottom:10px solid #000; }`.

Y Grid makes the painting; Flexbox hangs it on the wall

```
body {  
  margin: 0;  
  min-height: 100vh;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

Local attempt - two remaining corrections

1. Add the body centering rule.
2. Change final row from 20px to 22px for the exact 748px sum.

Source files remain unchanged. Selva should make these hands-on fixes.

Z Design-to-grid workflow

1	Draw every boundary over the reference.	6	Merge regions with <code>span</code> or <code>grid-area</code> .
2	Count tracks; remember tracks + 1 = lines.	7	Apply colours after geometry matches.
3	Write container width, height, tracks, gap.	8	Inspect track sizes in DevTools.
4	Add one div per visual region.	9	Prove tracks + gaps equal frame size.
5	Let auto-placement handle simple cells.	10	Center the finished component last.

DEBUGGING CLINIC - symptom, cause, fix

Read the property, the owner, and the axis. Grid bugs are often valid CSS with the wrong mental model.

AA The failures most likely to bite

Symptom	Likely cause	Fast check / fix
Blank grid; items exist	<code>fr</code> rows have no height to divide	Inspect container height; size container or children.
Columns respond, rows do not	<code>auto</code> axis asymmetry	Auto column fills width; auto row fits content.
Rule silently does nothing	misspelled property, often missing final <code>s</code>	Check Styles for ignored rule; spell <code>-rows</code> / <code>-columns</code> .
Item in wrong rectangle	counted cells instead of lines or swapped <code>grid-area</code> values	Enable line numbers; say row/column/row/column.
Item in correct sequence but too narrow	used <code>order</code> , forgot <code>span</code>	Separate sequence from size.
Fixed squares show mystery whitespace	tracks wider than the fixed children	Inspect track size; size the board or use stretch.
Extra items have odd height	implicit row fits content	Set <code>grid-auto-rows</code> .
Artwork is 2px off	track + gap arithmetic mismatch	Sum every track and every gap.

AB Property ownership - ask before editing

Visual question	Owner	Typical property
How many columns?	container	<code>grid-template-columns</code>
Where does this box start/end?	item	<code>grid-column</code> , <code>grid-row</code> , <code>grid-area</code>
How much space is between cells?	container	<code>gap</code>
How tall are newly created rows?	container	<code>grid-auto-rows</code>
Which overlapping item is on top?	item	<code>z-index</code>
Where is text inside one cell?	item as a new layout container	<code>display:flex</code> + alignment

AC Reusable patterns you will actually type

<pre>/* Equal responsive columns */ .cards { display:grid; grid-template-columns:repeat(3,1fr); gap:1rem; } /* Fixed sidebar + fluid content */ .layout { display:grid; grid-template-columns:250px 1fr; }</pre>	<pre>/* Flexible tracks with a floor */ .gallery { display:grid; grid-template-columns: repeat(3,minmax(200px,1fr)); } /* Full-width header/footer */ header, footer { grid-column:1 / -1; }</pre>
---	---

AD Three questions before writing CSS

1. Map or address? Container track definition, or item placement?	2. Where or how many? Line destination, or <code>span</code> count?	3. Fixed or flexible? px/rem, auto, fr, or minmax?
---	---	--

Selva's full debugging sentence
Check the owner, check the axis, check whether I am positioning or sizing, then let DevTools show the computed tracks.

RAPID-FIRE SELF TEST

Cover the answer columns. Target: all 20 in under 4 minutes, then write the two shorthands from memory.

#	Question	Answer	#	Question	Answer
1	Flexbox vs Grid dimensions	1D vs 2D	11	grid-area order	row/col/row/col
2	display:grid goes on?	container	12	grid-template order	rows / columns
3	3 columns have how many lines?	4	13	order changes?	visual queue
4	fr means?	share of available space	14	minmax(200px, 500px)	floor 200, ceiling 500
5	Auto column does?	fills leftover width	15	repeat(8, 1fr)	eight equal tracks
6	Auto row does?	fits content height	16	Blank fr rows: first check?	container height
7	Too many items create?	implicit tracks	17	Grid line colour trick	parent background + gap
8	Control implicit row height	grid-auto-rows	18	Mondrian final row	22px
9	Final line from far edge	-1	19	Grid Garden completed?	28/28
10	Start 2, cover 3 columns	2 / span 3	20	Show exact track pixels	DevTools Show track sizes

AE Honest remaining-practice checklist

Work	Status	Next action
EX 10.0 Chessboard	10/10	Rebuild from memory once.
EX 10.1 Grid Sizing Test	Not attempted by Selva	Type the five container rules before checking notes.
EX 10.2 exercise 2	One correction	Add astronaut's two-column span yourself.
Grid Garden	28/28 verified	Reset and repeat without this solution table.
EX 10.3 Mondrian	Near complete	Add body centering and use a 22px final row.

AF Notes, tasks, and references

Notes and practice files

- concept_grid_display.md + grid_display_tasks.md
- concept_grid_sizing.md + grid_sizing_tasks.md
- concept_grid_placement.md + grid_placement_tasks.md
- concept_grid_garden.md + grid_garden_tasks.md
- concept_mondrian_project.md + mondrian_project_tasks.md

Practice and lookup

- Grid Garden - appbrewery.github.io/gridgarden/
- Grid vs Flexbox - appbrewery.github.io/grid-vs-flexbox/
- Grid Sizing - appbrewery.github.io/grid-sizing/
- MDN CSS Grid Layout + Chrome DevTools Grid overlay

The five sentences to leave with

1. Flexbox aligns a line; Grid coordinates rows and columns.
2. The container draws the map; each item may choose an address.
3. Count lines for where, use span for how many.
4. Fixed tracks and gaps eat first; fr shares the leftovers.
5. When Grid looks wrong, check owner, axis, size, spelling, then the DevTools overlay.